

Inteligencia Artificial y Mini-Robots

Taller 6

Presentado por:

Cristian Camilo García Gamboa - crigarciaga@unal.edu.co

Mateo Becerra Porras - mabecerrap@unal.edu.co

Santiago Ospina Contreras - saospinaco@unal.edu.co

Profesor:

José J. Martínez P.

jmcursosingenieria111@gmail.com

Fecha de entrega:

06 de Octubre de 2026



Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
2026 II

1. Considere la figura 6.1. Conforme un dataset con resultados de 50.000 multiplicaciones de matrices 2X2, generadas aleatoriamente con números enteros entre -20 y 20. Una vez entrenado el modelo, úselo con 10 ejemplos y compare los resultados con los que se producen al realizar la operación analíticamente. Haga un cálculo de que es más costoso computacionalmente.

Generar un conjunto de datos de 50.000 ejemplos de multiplicación de matrices de dimensión 2x2. Cuyos elementos son números enteros aleatorios entre -20 y 20. Posteriormente, entrenar una red neuronal para aproximar el resultado de la multiplicación, evaluar su desempeño con 10 ejemplos nuevos y comparar su costo computacional con el método analítico tradicional.

En la programación tradicional, se introducen los datos y se ejecutan las reglas matemáticas definidas por el programador. En este caso, se aplica directamente la multiplicación de matrices.

Sean las matrices:

$$A = \begin{pmatrix} a & b & c & d \end{pmatrix}$$

$$B = \begin{pmatrix} e & f & g & h \end{pmatrix}$$

Su producto es:

$$AB = \begin{pmatrix} ae + bf & af + bg + ch & ag + bh + cd & ah + bg + dh \end{pmatrix}$$

En el aprendizaje automático, por el contrario, se proporcionan ejemplos de entrada y sus resultados correctos. La red neuronal ajusta sus parámetros durante el entrenamiento para aproximar la relación entre las entradas y las salidas.

Código en Python

Se utilizará NumPy para generar los datos y scikit-learn para entrenar una red neuronal multicapa.

```
Python
import numpy as np

import time

from sklearn.neural_network import MLPRegressor

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error

# Reproducibilidad de los resultados
```

```

np.random.seed(42)

# -----
# 1. Generar 50.000 ejemplos de multiplicacion
# -----

# Cada matriz tiene cuatro elementos entre -20 y 20
A = np.random.randint(-20, 21, size=(50000, 2, 2))
B = np.random.randint(-20, 21, size=(50000, 2, 2))

# Calcular las respuestas correctas
C = np.matmul(A, B)

# La red recibe 8 entradas y predice 4 salidas
X = np.concatenate(
    (A.reshape(50000, 4), B.reshape(50000, 4)),
    axis=1
)

Y = C.reshape(50000, 4)

# Normalizar entradas y salidas para facilitar
# el entrenamiento de la red neuronal
X = X / 20.0
Y = Y / 800.0

# Separar datos de entrenamiento y prueba
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=10, random_state=42
)

```

```

# -----

# 2. Crear y entrenar el modelo

# -----

modelo = MLPRegressor(

    hidden_layer_sizes=(64, 64),

    activation="relu",

    solver="adam",

    max_iter=300,

    random_state=42,

    early_stopping=True,

    n_iter_no_change=15

)

inicio_entrenamiento = time.perf_counter()

modelo.fit(X_train, Y_train)

fin_entrenamiento = time.perf_counter()

print("Tiempo de entrenamiento:",

      fin_entrenamiento - inicio_entrenamiento, "segundos")

# -----

# 3. Evaluar 10 ejemplos

# -----

inicio_prediccion = time.perf_counter()

predicciones = modelo.predict(X_test) * 800.0

fin_prediccion = time.perf_counter()

```

```

# Recuperar las matrices originales

A_test = (X_test[:, :4] * 20).astype(int).reshape(-1, 2, 2)
B_test = (X_test[:, 4:] * 20).astype(int).reshape(-1, 2, 2)

print("\nRESULTADOS DE LOS 10 EJEMPLOS")

for i in range(10):

    resultado_analitico = A_test[i] @ B_test[i]

    resultado_modelo = np rint(predicciones[i]).astype(int).reshape(2, 2)

    print(f"\nEjemplo {i + 1}")
    print("Matriz A:\n", A_test[i])
    print("Matriz B:\n", B_test[i])
    print("Resultado analitico:\n", resultado_analitico)
    print("Prediccion del modelo:\n", resultado_modelo)
    print("Coincidencia exacta:",
          np.array_equal(resultado_analitico, resultado_modelo))

# -----

# 4. Medir el error del modelo

# -----

Y_test_original = Y_test * 800.0

error = mean_absolute_error(
    Y_test_original.flatten(),
    predicciones.flatten()
)

print("\nError absoluto medio:", error)

```

```

print("Tiempo de prediccion de 10 ejemplos:",
      fin_prediccion - inicio_prediccion, "segundos")

# -----
# 5. Medir el costo de la operacion tradicional
# -----

inicio_tradicional = time.perf_counter()

for i in range(10):
    resultado = A_test[i] @ B_test[i]

fin_tradicional = time.perf_counter()

print("Tiempo analitico para 10 ejemplos:",
      fin_tradicional - inicio_tradicional, "segundos")

```

Análisis de resultados

Al ejecutar el programa se obtienen tres resultados relevantes:

- El tiempo que tarda la red neuronal en entrenarse con los 49.990 ejemplos de entrenamiento.
- Las predicciones del modelo para los 10 ejemplos de prueba, comparadas con los resultados exactos.
- El tiempo de predicción y el tiempo requerido por la multiplicación analítica tradicional.

Las predicciones de la red neuronal son aproximaciones. Debido a que la multiplicación de matrices es una operación matemática exacta y el modelo es una aproximación aprendida, no se garantiza que las 10 predicciones sean idénticas a los resultados analíticos.

Comparación del costo computacional

Para multiplicar dos matrices 2x2, el método tradicional necesita ocho multiplicaciones y cuatro sumas. Por tanto, su costo es constante para matrices de ese tamaño, y se expresa como:

$$O(1)$$

La red neuronal, en cambio, necesita realizar numerosas operaciones durante el entrenamiento. Para una red con capas de 8 entradas, 64 neuronas, 64 neuronas y 4 salidas, cada predicción requiere aproximadamente:

$$8(64) + 64(64) + 64(4) = 4864$$

multiplicaciones de pesos, sin contar las operaciones adicionales de activación y sesgo.

Aspecto	Método tradicional	Aprendizaje automático
Entrenamiento	No requiere	Sí requiere
Exactitud	Exacta, salvo errores numéricos	Aproximada
Costo para una matriz 2×2	Muy bajo	Mayor por predicción
Utilidad	Operaciones matemáticas conocidas	Problemas donde se aprende una relación a partir de datos

Conclusión: para multiplicar matrices de tamaño 2x2, la programación tradicional es más eficiente y exacta. Entrenar una red neuronal para resolver esta operación no representa una ventaja computacional, aunque permite estudiar cómo un modelo aprende una relación a partir de ejemplos.

De los últimos 4 ejercicios haga 3.

3. Estudie el algoritmo de K- Nearest, con todo detalle mejore su documentación y con base en el haga cambios para una aplicación.

K-Nearest Neighbors, también conocido como KNN o método de los k vecinos más cercanos, es un algoritmo de aprendizaje supervisado que puede utilizarse para clasificación y regresión. Su principio de funcionamiento consiste en buscar los k ejemplos del conjunto de entrenamiento que se encuentran más cerca del nuevo dato. En clasificación, se asigna al nuevo ejemplo la clase que aparece con mayor frecuencia entre sus vecinos.

La distancia euclidiana entre dos observaciones se calcula mediante:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Donde:

- x representa el nuevo ejemplo.
- y representa un ejemplo del conjunto de entrenamiento.

- n es el número de características.
- $d(x, y)$ es la distancia entre ambos ejemplos.

Ventajas

- Es sencillo de entender e implementar.
- No necesita construir un modelo complejo durante el entrenamiento.
- Puede trabajar con problemas de clasificación y regresión.

Desventajas

- Las predicciones pueden ser lentas con conjuntos de datos grandes.
- Es sensible a la escala de las variables.
- El valor de k puede afectar significativamente el resultado.

Código en Python

El siguiente programa utiliza el conjunto Adult Census Income, disponible mediante OpenML. Los datos se originaron en registros del censo estadounidense y contienen características como edad, educación, ocupación y horas trabajadas por semana. La variable objetivo clasifica los ingresos en dos categorías.

Python

Instalar previamente:

pip install numpy pandas scikit-learn

import pandas as pd

from sklearn.datasets import fetch_openml

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.impute import SimpleImputer

from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report

-----

1. Cargar los datos del censo

```
# -----

datos = fetch_openml(
    name="adult",
    version=2,
    as_frame=True
)

X = datos.data
y = datos.target

# Seleccionar una muestra para facilitar la ejecucion
muestra = X.sample(n=10000, random_state=42).index
X = X.loc[muestra]
y = y.loc[muestra]

# -----

# 2. Identificar tipos de variables

# -----

variables_numericas = X.select_dtypes(
    include=["number"]
).columns.tolist()

variables_categoricas = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

# Preparar las variables numericas
transformacion_numerica = Pipeline([
```

```

        ("imputacion", SimpleImputer(strategy="median")),

        ("escalado", StandardScaler())

    ])

# Preparar las variables categoricas
transformacion_categorica = Pipeline([

    ("imputacion", SimpleImputer(strategy="most_frequent")),

    ("codificacion", OneHotEncoder(handle_unknown="ignore"))

])

# Aplicar el procesamiento segun el tipo de variable
preprocesamiento = ColumnTransformer([

    ("numericas", transformacion_numerica, variables_numericas),

    ("categoricas", transformacion_categorica, variables_categoricas)

])

# -----

# 3. Separar entrenamiento y prueba

# -----

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.20,

    random_state=42,

    stratify=y

)

# -----

# 4. Crear y entrenar KNN

# -----

```

```

modelo_knn = Pipeline([
    ("preprocesamiento", preprocesamiento),
    ("clasificador", KNeighborsClassifier(n_neighbors=5))
])

modelo_knn.fit(X_train, y_train)

# -----
# 5. Realizar predicciones
# -----

predicciones = modelo_knn.predict(X_test)

print("Exactitud del modelo KNN:",
      accuracy_score(y_test, predicciones))

print("\nReporte de clasificacion:")
print(classification_report(y_test, predicciones))

# Ejemplo: clasificar una persona del conjunto de prueba
ejemplo = X_test.iloc[[0]]
prediccion = modelo_knn.predict(ejemplo)

print("Características del ejemplo:\n", ejemplo)
print("Clase predicha:", prediccion[0])
print("Clase real:", y_test.iloc[0])

```

Explicación del código

Se carga el conjunto de datos y se seleccionan 10.000 observaciones para reducir el costo computacional. Se identifican las variables numéricas y categóricas. Las variables numéricas

se escalan para evitar que las de mayor magnitud dominen el cálculo de distancias. Las variables categóricas se convierten a una representación numérica mediante codificación *One-Hot*. Se separan los datos en entrenamiento y prueba. Se configura KNN con $k = 5$, es decir, cinco vecinos cercanos. Se calcula la exactitud comparando las predicciones con las categorías reales.

Conclusión

KNN permite clasificar observaciones según su similitud con ejemplos previamente conocidos. Su rendimiento depende de la calidad de los datos, del procesamiento de las variables y de la selección de k . En esta aplicación, el algoritmo intenta predecir una categoría de ingresos a partir de características personales y laborales. La exactitud obtenida debe verificarse al ejecutar el programa y no debe interpretarse como una relación causal entre las características y los ingresos.

4. Estudie el algoritmo de árboles de decisión, con todo detalle mejore su documentación y con base en el haga cambios para una aplicación.

Este programa estudia el funcionamiento de los árboles de decisión, documenta sus componentes e implementa una aplicación de clasificación con el mismo conjunto de datos del censo.

Un árbol de decisión es un algoritmo de aprendizaje supervisado que construye una estructura jerárquica de reglas para clasificar datos o predecir valores numéricos.

El árbol está compuesto por:

- Nodo raíz: contiene todos los datos iniciales.
- Nodos internos: representan decisiones o condiciones sobre las variables.
- Ramas: representan los resultados de esas condiciones.
- Hojas: contienen la clase o el valor predicho.

Por ejemplo, un árbol podría dividir los datos según el nivel educativo y las horas trabajadas por semana para estimar la categoría de ingresos.

Uno de los criterios que puede utilizarse para construir el árbol es el índice de Gini:

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

Donde

p_i es la proporción de observaciones pertenecientes a la clase.

i y c es el número total de clases.

Un índice de Gini igual a cero indica que todas las observaciones de un nodo pertenecen a la misma clase.

Ventajas

- Permite interpretar las decisiones tomadas por el modelo.
- Puede representar relaciones no lineales.
- No requiere normalizar las variables numéricas.

Desventajas

- Puede sobreajustarse a los datos de entrenamiento.
- Es sensible a cambios en los datos.
- Un árbol muy profundo puede resultar difícil de interpretar.

Código en Python

Este programa utiliza el mismo conjunto de datos del ejercicio anterior, por lo que no es necesario descargar otro conjunto de datos si ambos ejercicios se ejecutan en el mismo entorno.

Python

```
from sklearn.datasets import fetch_openml

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder

from sklearn.impute import SimpleImputer

from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.metrics import accuracy_score, classification_report


# -----

# 1. Cargar los datos

# -----


datos = fetch_openml(

    name="adult",

    version=2,
```

```

        as_frame=True
    )

X = datos.data
y = datos.target

# Seleccionar una muestra reproducible

muestra = X.sample(n=10000, random_state=42).index
X = X.loc[muestra]
y = y.loc[muestra]

# -----

# 2. Preparar las variables

# -----

variables_numericas = X.select_dtypes(
    include=["number"]
).columns.tolist()

variables_categoricas = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

preprocesamiento = ColumnTransformer([
    (
        "numericas",
        SimpleImputer(strategy="median"),
        variables_numericas
    ),
    (

```

```

        "categoricas",

        Pipeline([

            ("imputacion",

             SimpleImputer(strategy="most_frequent")),

            ("codificacion",

             OneHotEncoder(handle_unknown="ignore"))

        ]),

        variables_categoricas

    )

1)

```

```

# -----
# 3. Separar los datos
# -----

```

```

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.20,

    random_state=42,

    stratify=y

)

```

```

# -----
# 4. Crear el arbol de decision
# -----

```

```

modelo_arbol = Pipeline([

    ("preprocesamiento", preprocesamiento),

    (

        "clasificador",

```

```

        DecisionTreeClassifier(

            criterion="gini",

            max_depth=5,

            min_samples_leaf=10,

            random_state=42

        )

    )

l)

# Entrenar el modelo

modelo_arbol.fit(X_train, y_train)

# -----

# 5. Evaluar el modelo

# -----

predicciones = modelo_arbol.predict(X_test)

print("Exactitud del arbol:",

      accuracy_score(y_test, predicciones))

print("\nReporte de clasificacion:")

print(classification_report(y_test, predicciones))

# -----

# 6. Mostrar las reglas aprendidas

# -----

nombres = modelo_arbol.named_steps[

    "preprocesamiento"

```

```

].get_feature_names_out()

arbol = modelo_arbol.named_steps["clasificador"]

reglas = export_text(
    arbol,
    feature_names=list(nombres),
    max_depth=5
)

print("\nReglas del arbol de decision:")
print(reglas)

```

Explicación del código

El programa carga los datos, separa las variables numéricas de las categóricas y realiza las transformaciones necesarias para que el algoritmo pueda utilizarlas.

Posteriormente, crea un árbol de decisión con los siguientes parámetros:

- `criterion="gini"`: utiliza el índice de Gini para seleccionar las divisiones.
- `max_depth=5`: limita la profundidad a cinco niveles para reducir el sobreajuste.
- `min_samples_leaf=10`: exige al menos diez observaciones en cada hoja.

Después de entrenar el árbol, se realizan predicciones sobre el conjunto de prueba y se calcula la exactitud. Finalmente, `export_text` presenta las reglas aprendidas en forma de texto, lo que permite examinar cómo el modelo divide las observaciones.

Conclusión

Los árboles de decisión permiten construir reglas comprensibles a partir de datos. En este caso, se emplean para clasificar las categorías de ingresos utilizando información censal. La profundidad máxima y el número mínimo de observaciones por hoja ayudan a controlar el sobreajuste. Una ventaja frente a KNN es que, una vez entrenado, el árbol puede realizar predicciones recorriendo una secuencia de condiciones sin calcular distancias respecto a todas las observaciones de entrenamiento.

5. Estudie el algoritmo de Bayes ingenuo y haga un ejemplo bien documentado.

El objetivo es estudiar el algoritmo de Bayes ingenuo, explicar su fundamento matemático e implementar un ejemplo documentado de clasificación de ingresos utilizando datos derivados del censo de Estados Unidos.

Bayes ingenuo, conocido también como *Naive Bayes*, es un algoritmo de clasificación basado en el teorema de Bayes.

El teorema de Bayes se expresa de la siguiente manera:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Donde:

$P(C | X)$: probabilidad de pertenecer a una clase

C dadas las características X

$P(X | C)$: probabilidad de observar las características si el ejemplo pertenece a la clase.

$P(C)$: probabilidad previa de la clase.

$P(X)$: probabilidad de observar las características.

Se denomina ingenuo porque asume que las características son condicionalmente independientes entre sí, dada la clase. Aunque esta suposición no siempre se cumple en los datos reales, el algoritmo puede funcionar bien en numerosos problemas de clasificación.

Existen diferentes variantes. Para este ejercicio se utilizará Gaussian Naive Bayes, que modela las variables numéricas mediante distribuciones gaussianas y puede trabajar con variables categóricas después de convertirlas en indicadores numéricos.

Ventajas

- Es relativamente sencillo de implementar.
- Su entrenamiento suele ser rápido.
- Puede funcionar bien con conjuntos de datos de muchas dimensiones.

Desventajas

- La independencia condicional puede no representar adecuadamente los datos.
- Las probabilidades estimadas pueden ser poco precisas.
- Su rendimiento depende de las características y de las suposiciones del modelo.

Código en Python

Se utiliza el mismo conjunto de datos del censo. Para que el algoritmo sea compatible con las variables categóricas, se transforman mediante codificación One-Hot y se utiliza una matriz densa.

Python

```
from sklearn.datasets import fetch_openml

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.impute import SimpleImputer

from sklearn.pipeline import Pipeline

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report


# -----

# 1. Cargar los datos

# -----


datos = fetch_openml(
    name="adult",
    version=2,
    as_frame=True
)

X = datos.data
y = datos.target


# Seleccionar 10.000 observaciones

muestra = X.sample(n=10000, random_state=42).index

X = X.loc[muestra]
y = y.loc[muestra]
```

```

# -----

# 2. Identificar tipos de variables

# -----

variables_numericas = X.select_dtypes(
    include=["number"]
).columns.tolist()

variables_categoricas = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

# Transformar variables numericas

transformacion_numerica = Pipeline([
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", StandardScaler())
])

# Transformar variables categoricas

transformacion_categorica = Pipeline([
    ("imputacion",
     SimpleImputer(strategy="most_frequent")),
    ("codificacion",
     OneHotEncoder(handle_unknown="ignore"))
])

# sparse_threshold=0 fuerza una matriz densa,
# necesaria para GaussianNB

preprocesamiento = ColumnTransformer(

```

```

[
    (
        "numericas",
        transformacion_numerica,
        variables_numericas
    ),
    (
        "categoricas",
        transformacion_categorica,
        variables_categoricas
    )
],
sparse_threshold=0
)

# -----
# 3. Separar entrenamiento y prueba
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# -----
# 4. Crear y entrenar el clasificador
# -----

```

```

modelo_bayes = Pipeline([
    ("preprocesamiento", preprocesamiento),
    ("clasificador", GaussianNB())
])

modelo_bayes.fit(X_train, y_train)

# -----
# 5. Realizar las predicciones
# -----

predicciones = modelo_bayes.predict(X_test)

print("Exactitud de Bayes ingenuo:",
      accuracy_score(y_test, predicciones))

print("\nReporte de clasificacion:")
print(classification_report(y_test, predicciones))

# -----
# 6. Calcular probabilidades para un ejemplo
# -----

ejemplo = X_test.iloc[[0]]

clase_predicha = modelo_bayes.predict(ejemplo)
probabilidades = modelo_bayes.predict_proba(ejemplo)

print("\nEjemplo seleccionado:")
print(ejemplo)

```

```
print("Clase predicha:", clase_predicha[0])

print("\nProbabilidades por clase:")

for clase, probabilidad in zip(
    modelo_bayes.classes_,
    probabilidades[0]
):
    print(clase, ":", round(probabilidad, 4))
```

Explicación del código

El programa se divide en seis etapas:

1. Carga de datos: obtiene las características personales y laborales y la categoría de ingresos.
2. Preprocesamiento: completa los valores faltantes, escala las variables numéricas y convierte las categóricas a valores numéricos.
3. División: utiliza el 80 % de los datos para entrenamiento y el 20 % para evaluación.
4. Entrenamiento: calcula las distribuciones necesarias para estimar las probabilidades de cada clase.
5. Predicción: determina la categoría más probable para cada observación.
6. Estimación de probabilidades: muestra las probabilidades que el modelo asigna a cada clase para un ejemplo concreto.

Conclusión

Bayes ingenuo permite realizar clasificaciones utilizando probabilidades y el teorema de Bayes. Su principal ventaja es que el entrenamiento suele ser rápido, incluso cuando se trabaja con muchas características. Sin embargo, la suposición de independencia entre variables puede limitar su capacidad predictiva. En este ejemplo, se aplica para estimar la categoría de ingresos a partir de características socioeconómicas, sin que las predicciones deban interpretarse como relaciones causales.

Comparación general de los algoritmos

Característica	KNN	Árbol de decisión	Bayes ingenuo
Fundamento	Distancia entre ejemplos	Reglas y divisiones	Probabilidad condicional
Entrenamiento	Muy sencillo	Construcción del árbol	Estimación de probabilidades
Predicción	Puede ser costosa con muchos datos	Generalmente rápida	Generalmente rápida
Interpretabilidad	Baja o moderada	Alta	Moderada
Escalado de variables	Importante	Generalmente innecesario	Depende de la variante
Riesgo principal	Sensibilidad a distancias y escala	Sobreajuste	Suposición de independencia

Conclusión general del taller

Los ejercicios permiten distinguir entre la programación tradicional y diferentes métodos de aprendizaje automático.

En la multiplicación de matrices, el método analítico es superior porque existe una solución matemática conocida, exacta y computacionalmente económica. Entrenar una red neuronal para imitar esta operación introduce un costo innecesario.

En cambio, los algoritmos KNN, árboles de decisión y Bayes ingenuo permiten estudiar problemas de clasificación en los que se busca encontrar patrones en datos existentes. Cada algoritmo aborda el problema desde una perspectiva diferente: similitud entre observaciones, reglas jerárquicas o estimación de probabilidades.